Ingegneria del Software — Soluzione del Tema 10/07/2023

Esercizio 1

Si consideri la seguente classe immutabile `Message`. Ogni messaggio ha un contenuto, un mittente, un destinatario e un timestamp. Mittente e destinatario sono individuati univocamente da un identificatore, per semplicità un long. Ancora per semplicità, il timestamp è un long (anziché un `Date`).

```
public /*@ pure @*/ class Message {
  public Message(byte[] content, long recipientID, long senderID, long timestamp) {...}
  public byte[] content() {...}    // returns the content of this
  public long recipientID() {...}  // returns the recipientID of this
  public long senderID() {...}     // returns the senderID of this
  public long timestamp() {...}    // returns the timestamp of this
}
```

La seguente classe `MessageManager` descrive un gestore di messaggi. È possibile inviare ed estrarre messaggi, vedere elenchi dei messaggi e degli utenti; in particolare, è possibile fondere un altro gestore messaggi con quello corrente.

```
public class MessageManager {
  // Builds a new empty manager
  public MessageManager() {...}

  // Returns the set of all valid ids, i.e., ids of recipients or senders of some message in the past.
  public /*@ pure @*/ Set<Long> allIds() {...}

  // Returns all messages yet to be delivered to the recipient having the given recipientId,
  // from the oldest to the newest.
  // Returns an empty list if the recipient is unknown or if there is no message to be delivered it.
  public /*@ pure @*/ List<Message> toId(long recipientId) {...}

  // Returns all messages, yet to be delivered, sent from the sender having the given senderId.
  // Throws an UnknownIdException if the sender is unknown.
  public /*@ pure @*/ Set<Message> fromId(long senderId) throws UnknownIdException {...}

  // Inserts a new message in the manager, with the timestamp at the instant of the call.
  public void addMessage(byte[] content, long senderId, long recipientId) {...}

  // Returns and deletes the oldest message added for recipientId and not yet delivered.
  // Throws a NoMessageExpetion if no message is present.
  // Throws an UnknownIDException if the id is unknown.
  public Message deliverTo(long recipientId) throws NoMessageException, UnknownIDException{...}

  // Copies all messages and ids in the manager other to this.
  // The manager other is unchanged.
  public void merge(MessageManager other) {...}
```

Domanda a)

Specificare in JML il metodo puro `fromId`.

Soluzione

```
//@ ensures allIds().contains(senderId) && \result!=null &&
//@ (\forall long recipientId; allIds().contains(recipientId);
//@   (\forall Message m; toId(recipientId).contains(m) && m.senderID()==senderId;
//@      \result.contains(m) ) ) &&
//@ (\forall Message m; \result.contains(m);
//@    m.senderID()==senderId && allIds.contains(m.recipientId()) &&
//@    toId(m.recipientId()).contains(m) )
//@
//@ signals(UnknownIDException e) !allId().contains(senderId);
public /*@ pure @*/ Set<Message> fromId(long senderId) throws UnknownIdException {...}
```

Domanda b)

Specificare in JML metodo `merge`.

Soluzione

Occorre dire che `other` al termine dell'invocazione è immutato, mentre this può cambiare. Al termine dell'invocazione, `allIds` deve contenere tutti e soli gli id precedentemente presenti in `this` oppure in `other`. Le liste dei messaggi ritornata da `toId` devono contenere tutti e soli i messaggi precedentemente ritornati invocando `toId` in `this` o in `other`. Infine, le liste dei messaggi ritornate da `toId` devono essere in ordine di timestamp.

```
//@ requires other!=null;
//@ ensures
//@ (\forall long id; ;
//@    allIds().contains(id) <==> (\old(allIds()).contains(id) || \old(other.allIds()).contains(id)) ) &&
//@ (\forall long recipientId; ;
```

```
//@    toId(recipientId).containsAll(\old(toId(recipientId))) &&
//@    toId(recipientId).containsAll(\old(other.toId(recipientId))) &&
//@    toId(recipientId).size() == \old(toId(recipientId)).size() + other.toId(recipientId).size()) &&
//@    (\forall int i; 0<=i && i<toId(recipientId).size()-1;
//@       toId(recipientId).get(i).timestamp() <= toId(recipientId).get(i+1).timestamp()) );
//@ other.allIds().containsAll(\old(other.allIds()) && other.allIds().size()==\old(other.allIds().size())  &&
//@ (\forall long id; ;
//@    other.toId(id).size()==\old(other.toId(id).size())) &&
//@    (\forall int i; i>=0 && i<other.toId(id).size();
//@       other.toId(id).get(i)==\old(other.toId(id).get(i)) ) )
public void merge(MessageManager other) {...}
```

Si noti che non è necessario specificare come cambia il metodo `fromId()`, in quanto la sua specifica già lo descrive in funzione degli altri due metodi puri. Si noti inoltre che messaggi inviati tramite gestori diversi potrebbero avere lo stesso timestamp.

Domanda c)

Si consideri la seguente semplice implementazione di `MessageManager`.

```
public class MessageManager {
  private Set<Long> validIds;
  private List<Message> pendingMessages;
  ...
}
```

I messaggi sono inseriti in ordine di arrivo nella `List`, aggiornando anche `validIds` se necessario. Una volta inviati (mediante invocazioni di `deliverTo`) i messaggi sono cancellati da `pendingMessages`. Scrivere in JML l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

Occorre dichiarare che i due attributi non sono nulli e non contengono valori nulli. Gli id dei sender e dei recipient dei messaggi in `pendingMessages` sono contenuti in `validIds`. Si noti che `validIds` potrebbe contenere degli id a cui non corrispondono messaggi (in quanto la specifica prevede di raccogliere tutti gli id per cui c'e' stato un messaggio in passato). Occorre infine specificare che i messaggi in `pendingMessages` non sono duplicati e sono in ordine di timestamp.

RI:

```
//@ private invariant validIds!=null && pendingMessages!=null &&
//@ !validIds.contains(null) && !pendingMessages.contains(null) &&
//@ (\forall Message m; pendingMessages.contains(m);
//@    validIds.contains(m.senderID()) && validIds.contains(m.recipientID()) ) &&
//@ (\forall int i; 0<=i && i<pendingMessages.size()-1;
//@    pendingMessages.get(i).timestamp() <= pendingMessages.get(i+1).timestamp() &&
//#    (\forall int j; i<j && j<pendingMessages.size();
//@       !pendingMessages.get(i).equals(pendingMessages.get(j))))
```

Per la funzione di astrazione occorre mettere in relazione il rep con i due metodi puri `allIds` e `toId` (il restante metodo puro `fromId` è specificato in funzione di questi).

AF:

```
//@ private invariant validIds.containsAll(allIds()) && validIds.size()==allIds.size() &&
//@ (\forall long recipientId; validIds.contains(recipientId);
//@    (\forall Message m; m.recipientId()==recipientId;
//@       toId(recipientId).contains(m) <==> pendingMessages.contains(m) )
//@    (\forall Message m1; toId(recipientId).contains(m1);
//@       (\forall Message m2; toId(recipientId).contains(m2);
//@          toId(recipientId).indexOf(m1) < toId(recipientId).indexOf(m2) <==>
//@          pendingMessages.indexOf(m1) < pendingMessages.indexOf(m2) ) ) )
```

Domanda d)

Si consideri una classe `ClearableMessageManager` che aggiunge alla classe `MessageManager` il metodo `clear` che cancella tutti gli id e tutti i messaggi pendenti. Può `ClearableMessageManager` essere definita come sottoclasse di `MessageManager` in accordo con il principio di sostituzione di Liskov? Si motivi la risposta.

Soluzione

`ClearableMessageManager` non può essere definita come sottoclasse di `MessageManager` in quanto violerebbe la proprietà evolutiva che richiede che gli id di tutti i mittenti e destinatari di messaggi passati siano conservati (ritornati da future chiamate di `allIds`).

Esercizio 2

Si consideri la seguente classe `Point` che rappresenta un punto nello spazio cartesiano.

```
public class Point {
  private double myX;
  private double myY;

  public synchronized void setX(double x) { myX = x; }
  public synchronized void setY(double y) { myY = y; }

  public synchronized void setXY(double x, double y) throws InterruptedException {
    while( (x+y)<0 ) wait();
    notifyAll();
    myX = x; myY = y;
  }
}
```

Domanda a)

Il metodo `setXY` è correttamente sincronizzato (ovvero non da luogo a conflitti nell'accesso alla memoria e non genera deadlock)? Motivare la propria risposta.

Soluzione

Il metodo da luogo ad un deadlock quando invocato con due valori la cui somma sia negativa. Il ciclo `while(...)` sospende infatti il chiamante fino a quando la condizione `(x+y)<0` è verificata, ma essendo la condizione data sul valore dei parametri, questi non possono mai essere cambiati da altri thread, con il risultato che il chiamante resta per sempre sospeso.

Domanda b)

Si scriva il metodo `boolean compare(Point other)` che ritorna `true` se e solo se il punto `other` ha le stesse coordinate del punto `this`. Si garantisca la correttezza dell'implementazione anche in presenza di thread multipli.

Soluzione

```
public boolean compare(Point other) {
  double x,y;
  synchronized(other) {
    x = other.myX;
    y = other.myY;
  }
  synchronized(this) {
    return x==myX && y==myY;
  }
}
```

Domanda c)

Si scriva il metodo `void printOncePerSecond(int numTimes)` che, in un thread separato rispetto al chiamante, stampa per `numTimes` volte le coordinate del punto, una ogni secondo. Si faccia attenzione a garantire il corretto accesso alla memoria condivisa, senza bloccare i thread oltre il tempo minimo indispensabile.

Si ricorda l'esistenza del metodo statico `Thread.sleep(millis)` che sospende il thread corrente per il numero indicato di millisecondi.

Soluzione

```
public void printOncePerSecond(int numTimes) {
  new Thread(() -> {
      for(int i=0; i<numTimes; i++) {
        synchronized(Point.this) {
          System.out.println("X: "+myX+" Y: "+myY);
        }
        try { Thread.sleep(1024); }
        catch(InterruptedException ex) { ex.printStackTrace(); }
      }
  }).start();
}
```

Si noti la necessità di acquisire il lock su `Point.this` per evitare conflitti in presenza di chiamate di altri metodi della classe `Point` (tutti sincronizzati su `this`).

Esercizio 3

Si considerino le seguenti classi di cui viene qui omessa l'ovvia implementazione:

```
public class Drawing {
  ...
  public List<Shape> getShapes() { ... }
  public String getName() { ... }
  public int getDiagonal() { ... }
}

public class Shape {
  ...
  public Color getColor() { ... }
}
```

Rispondere alle domande seguenti usando **esclusivamente** i costrutti della *programmazione funzionale* (*senza quindi usare while, if.. else, for,* ecc.)

Domanda a)

Completare l'implementazione del seguente metodo `printNameOfBiggest` che prende in ingresso una lista di dipinti (`Drawing`) e stampa il nome del dipinto avente la diagonale maggiore tra quelli nella lista. Se la lista è vuota, il metodo non stampa nulla. Se esistono più dipinti con la stessa diagonale, il metodo stampa comunque un solo nome (non viene specificato quale).

```
public static void printNameOfBiggest(List<Drawing> drawings) {
  ...
}
```

Soluzione

```
public static void printNameOfBiggest(List<Drawing> drawings) {
  drawings.stream()
    .max((d1, d2) -> Integer.compare(d1.getDiagonal(), d2.getDiagonal()))
    .map(Drawing::getName)
    .ifPresent(System.out::println);
}
```

Domanda b)

Completare l'implementazione del seguente metodo `getAllColors` che prende in ingresso una lista di dipinti (`Drawing`) e ritorna una lista con tutti i colori utilizzati in almeno uno dei dipinti della lista. La lista ritornata non deve contenere duplicati.

```
public static List<Color> getAllColors(List<Drawing> drawings) {
  return ...
}
```

Soluzione

```
public static List<Color> getAllColors(List<Drawing> drawings) {
  return drawings.stream()
    .flatMap(d -> d.getShapes().stream())
    .map(Shape::getColor)
    .distinct()
    .collect(Collectors.toList());
}
```

Esercizio 4

Si consideri il seguente metodo statico:

```
public static int test(int x, int y) {
  int k = 0;
  if(x>0) {
    if(y>0) {
      k += y;
    } else {
      k += -y;
    }
  }
  if(x==10) {
    k += 10;
  }
  return k;
}
```

Domanda a)

Si determini un insieme minimo di casi di test per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Servono almeno due casi di test al fine di entrare sia nel ramo `if` che nel ramo `else` dello statement condizionale all'interno del primo `if`. Ad esempio i due casi di test: `(10,1)` e `(10,0)`

Domanda b)

Si determini un insieme minimo di casi di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

Ai due casi precedenti si devono aggiungere i casi di test necessari a valutare false le due condizioni degli `if` "esterni". Basta un caso di test, ad esempio `(0,qualsiasi)`. In definitiva servono tre casi di test: `(10,1)`, `(10,0)` e `(0,qualsiasi)`

Domanda c)

Si determini un insieme minimo di casi di test per garantire la copertura dei cammini (path coverage).

Soluzione

In totale ci sono 5 cammini possibili, esercitati dai seguenti casi di test: `(10,1)`, `(5,1)`, `(10,0)`, `(5,0)` e `(0,qualsiasi)`.